Group 15, Video 2

Link to the Video:
https://mediaspace.msu.edu/media/CSE498+Spring+2026+-+Group15Video2+-+FinalVideo/1_tmd6ow1e

Script:

BSP-Dungeon (Paul)

The BSP-Dungeon generates a tree where each node represents a certain, randomly split sub-section of a 2d-grid, represented as a vector of strings, whose space will hold a certain type of room depending on the 'World Level'. It utilizes another class, RoomHolder.hpp, to randomly select a pre-made room from a certain pool of rooms of that corresponding world level to be put in that sub-section using LoadRoom(). The tree is represented as a vector of structs. Each node in the tree is represented as a struct, named 'BSPNode' that contains information of each sub-sections width, height, x-y coordinate location, and an integer value corresponding to the location of its left/right child in the vector. Nodes/sub-sections will continue to be recursively created and populated with rooms until it reaches a certain width/height threshold or the limit of splits set by m_iterations is met. This tree's info is then passed off into WorldGeneration to generate the world.

DungeonWorld- Overview (Joey)

DungeonWorld inherits from a class called worldbase which is the foundation of the game world. This gives DungeonWorld the tile grid and agent management it needs to run. For creating a level, any time a level needs to exist, DungeonWorld calls GenerateLevel(). This function calls a helper called LoadLevelData which checks a counter called mLevelnum and picks the right level type. Level one is the forest, then the cave, and then the castle. That configuration is stored in a smart pointer called mLevel. Then GenerateLevel creates a fresh worldgeneration object. mGeneration generates the dungeon layout, and loads it into the tile grid. When the player steps on an exit tile DoAction calls Advancelevel. That first calls despawndungeonagents which destroys every tracked enemy inm the world. Then clear level wipes the dungeon data from mGeneration, mLevelnum increments, and Generatelevel runs again for the new level.

DungeonWorld- Agent Interaction (Abigail)

DungeonWorld mainly interacts with agents through the function RunAgents(). In this function it loops through the enemy agents, meaning it ignores the interface and player agents, and each enemy selects an action using SelectAction(). The actions an agent can choose from are defined in ConfigAgent(), which maps movement and interact to specific action IDs. The selected action is then passed into DoAction(). In DoAction(), DungeonWorld checks whether the action is valid and then applies it. For movement, it calculates the new position and prevents invalid moves like going out of bounds or into walls. If the player agent is performing an action of type "interact", the function will check all tiles within 1 tile from the agent, for other agents. If an

enemy is within this range of the player, combat is triggered. The player can deal damage to the enemy and the amount dealt will be calculated. If the enemy survives after the player turn, it can retaliate and deal damage back. If the enemy is defeated, it is removed and the player is rewarded by gaining gold. If the player agent runs an "interact" action and no enemy is nearby, then the action will be ignored. DoAction() also handles other world updates, like giving the player loot when stepping on a loot tile and advancing to the next dungeon level when reaching an exit door, which resets the world and spawns new enemies. Overall, agents decide what they want to do, and then DungeonWorld controls whether it will happen and then updates the game state accordingly.

DungeonWorld- UI Interaction (Rachel)

One of the important things for our module to be able to do is to send data to the UI groups so that they can visually display the world that we generate. We do that the the DungeonWorld class, which you can see right here. DungeonWorld has a series of size_t member variables. Each variable is associated with a character value that the dungeon generates to denote a different tile type. Floor tiles, wall tiles, trap tiles, loot tiles, monster spawn tiles, door tiles, secret door tiles and exit door tiles each have a member variable and a unique character they are associated with. Expanding on that, each individual image file has its own member variable and character associated with it. So, there is a member variable for each type of floor tile, for each level. This one-to-one ratio allows for the UI groups to be able to quickly determine what image file they need to load for any given tile. By sending this data to the UI groups through GetGrid(), we can ensure that they are able to quickly and easily display the randomly generated levels.

LevelBase (Zhixiang)

LevelBase. There are two getter functions in LevelBase class, and because each level class has to use these two functions, it is better to make them virtual instead of just make the exact same functions into three different level classes and that is redundant. For each level there are ids for each room and its weight.